

data is raw facts with no context. ex number 42 sitting in a cell. **info** is data that's been processed and given meaning "the patient's temperature is 42°C"

Q:"Where does data come from / what constitutes data?"in the world there's no inherent thing called "data" - but there's nothing that isn't data either. The matter around us is raw data; the remarkable thing is that we can take essentially anything (flora, the air's composition, people) and "constitute" it as data by capturing its characteristics as columns and writing it down as tables. Those tables can then be normalized, related, queried. So data is an abstraction we impose on matter (including ourselves and our devices, plus metadata about all of it), and it keeps growing. The key sentence to land: an entity is just a set of characteristics/columns, and data is what you get when you record those characteristics - it's derived from the real world, not found pre-packaged in it.

Db handles access, structure, security, multiple people at once, and recovery after crashes.

file systems **4 pains**-structural dependence(change file's layout and every program that reads it breaks), data dependence, redundancy (the same fact copied in many files, so updates get inconsistent), "islands of info"(siloes files that don't agree). **categorize dbs** by no of users, by location (centralized/distributed/cloud), by use (**OLTP**(online transaction processing)=day-to-day "operational" transactions,many small fast read/write transactions, current data, normalized. vs **OLAP**(online analytical processing)="analytical",complex read-heavy queries over historical data, lives in a data warehouse, denormalized / star schema.e),by structure (relational, NoSQL, vector)

Special purpose db examples - Family-tree (genealogy) DB — Stores people and who their parents/children are. Interface: a clickable family tree. Special: it has to follow links up and down many generations, which normal tables do poorly. Stock market DB — Stores prices over time (time, stock, price). Interface: charts and a calculator for averages. Special: it's all time-based data and you mostly ask "what happened between these dates," with live updates. Bitcoin / blockchain DB — Stores a chain of transactions that can never be changed. Interface: a "block explorer" to look up a transaction or address. Special: data is add-only (no editing or deleting) and copied across many computers, with no single owner. Fingerprint DB — Stores fingerprint scans. Interface: a matcher that finds the closest fingerprints. Special: you ask "which is the closest match," not "which is exactly equal." Astronomy DB — Stores stars and galaxies with their sky positions and images. Interface: a sky map where you search an area of the sky. Special: huge amounts of data and you search by location in the sky, plus big images.

model is abstraction-you capture the parts of reality that matter and throw rest away. data model is agreed-upon shape of your data before you build anything.

Data model evolution: Hierarchical(tree - every record has exactly one parent, like org chart; rigid)→Network(graph-record can have many parents, like flight map; more flexible but complex)→Relational(everything is tables; simple and dominant)→ER(design notation for planning relational DBs)→Object-Oriented/Object-Relational→NoSQL→Vector

Data abstraction layers: External=each user's own partial view. Conceptual=unified,DBMS-independent design(this is where ER diagrams live).Internal=DBMS-specific logical schema.Physical=actual bytes on disk: files, blocks, indexes.

Why do we care?whole point is independence:you can change how data is physically stored (physical level) without breaking user's view(external level).That separation is what made databases robust.

ER Diagram:entity(noun-"thing") is rectangle and becomes table; attribute(property of that thing) is ellipse and becomes col;relationship (verb connecting things) is diamond

Attributes: simple vs composite (one attribute made of smaller pieces), single vs multivalued, derived (computed, not stored).Composite ex: address->(street,city,zip); color(rgb). Derived ex: age from birthdate

4 things that describe relationship:Connectivity -type: 1:1,1:M, or M:N.(A professor → many classes is 1:M.) called cardinality ratios Cardinality - exact (min, max) numbers written on the line like (1,4). Sometimes min is called modality, max is cardinality. #rows in table is also called cardinality

Participation -optional (minimum 0 you don't have to participate) or mandatory (minimum 1).

Degree -how many entities the relationship connects. # of cols

Unary recursive: COURSE is a prerequisite of COURSE (one entity related to itself), emp mentors emp.

Unary non-recursive: a single entity PERSON participating in a relationship like "can vote" / "is eligible"; account is active one entity, no second entity and no self-link.

Binary: AUTHOR writes BOOK, student enrolls in course, doc treats patient

Ternary: DOCTOR prescribes DRUG to PATIENT; SUPPLIER supplies PART to PROJECT; Teacher Alice teaches Mathematics to Class 10A.

Quaternary: DOCTOR prescribes DRUG to PATIENT at a PHARMACY the fact needs all four together.CUSTOMER buys PRODUCT from STORE using PAYMENT_METHOD

5-way:CUSTOMER buys PRODUCT from STORE using PAYMENT_METHOD on DATE; Student Bob takes Database Systems taught by Prof. Lee in Fall 2026 at the Main Campus.

weak entity(existence dependence) is one that can't exist on its own; **existence-dependent** on parent and inherits part of the parent's PK. Ex:DEPENDENT can't exist without EMPLOYEE; its PK is (emp_num, dep_name). test he gives: "can I exist apart from you?" If no, it's weak. an invoice LINE needs an INVOICE.

Weak relationship: the related entity's PK does not contain the parent's PK (the parent's key appears only as a non-key FK). Ex: a CLASS referencing COURSE via course_id as a plain foreign key -the class's own PK doesn't include course_id. (Strong/identifying relationship = the parent's key is part of the child's PK.)

Q: in computing, strong dependence of one component on another is considered "bad." Explain why, first for file-based data storage, then for existence-dependent (weak) entity. A: Strong coupling means a change in one place forces changes elsewhere and can break things. File storage: programs are structurally dependent on the file layout - change the format (add a field, reorder columns) and every program reading that file breaks. Weak entity: a DEPENDENT is existence-dependent on EMPLOYEE and carries the employee's key in its own PK; if the employee row is removed, the dependent can no longer exist -its fate is tied to the parent. In both cases tight dependence reduces independence and propagates failure.

bridge (composite) entity exists to resolve messy M:N relationship into two clean 1:M ones.Students and classes are M:N, so you insert ENROLLMENT in between, with composite key (stu_id, class_id) and room for extras like a grade.

EER(less redundancy, clean design than flat ER when things naturally specialise) just adds inheritance to ER:same "is-a" idea from object-oriented programming. You have general **supertype** and specialized **subtypes** and subtypes inherit supertype's columns. EMPLOYEE is supertype; PILOT, MECHANIC, ACCOUNTANT are subtypes. Each inherits emp_id and name, then adds its own columns (a pilot's license number). A pilot is an employee.

Hierarchy two ways: Disjointness: is an instance in at most one subtype(disjoint, ex ACCOUNT is checking or savings) or possibly several **overlapping**, ex PERSON who is both EMPLOYEE and STUDENT)?

Completeness:must every supertype instance belong to some subtype (total) or can it belong to none (partial)?

2 directions: specialization is top-down (split a general thing into specifics); **generalization** is bottom-up (notice several things share traits and merge them into a supertype).

union is a subtype tied to several diff supertypes(OWNER might be PERSON or COMPANY);

lattice is subtype with more than one supertype.

Functional dependency - idea behind keys and normalisation

functional dependency $X \rightarrow Y$ means "X determines Y": if you know X, there is exactly one Y. X is the determinant. This one idea powers both keys(key determines everything) and normalization. $STU_ID \rightarrow STU_NAME$: an ID gives you exactly one name. reverse isn't a dependency (many people share a name) $Stu_id, stu_lname \rightarrow gpa$ is not a full functional dependency cause determinant contains extra(unwanted) attr stu_lname $stu_lname, stu_fname \rightarrow gpa$ is full functional dependency assuming name, fname is unique

Keys types:

superkey is any set of columns that uniquely identifies a row (even with extra junk).

candidate key is a minimal superkey (no extra columns).

primary key is the candidate key you pick; it must be unique and not null. determines all the other (non-prime) attributes that's functional dependency / determination. **DISTINCT** = a SQL keyword that removes duplicate rows from a query's result (deduplication in the output). PK enforces uniqueness in the stored table; DISTINCT just removes duplicates in the output of a query

composite key is a primary key made of two or more columns.

foreign key is column that points at another table's primary key that's how tables link

Integrity rules:

Entity integrity:primary key can't be null and can't repeat (otherwise you can't tell rows apart).

Referential integrity: foreign key must either match real primary-key value somewhere, or be null; you can't point at something that doesn't exist.

nulls are dangerous/undesirable in relational tables because Ambiguous meaning: a NULL fee_paid could mean "not paid," "unknown," or "not entered yet" - three different truths stored as one blank, so queries and logic can't trust it. Breaks counts/logic & integrity: COUNT(col) and WHERE col <> 'x' silently skip NULLs, and a NULL where a value is required (e.g. a blank student ID on a registration) lets bad/incomplete rows in. Three-valued logic (true/false/unknown) makes results surprising.

Closure-Every relational-algebra operation takes table (or two) as input and produces a table as output so results are predictable and you can chain operations w/out breaking system i.e you stay inside the table. That self-similarity is closure. Even result with zero rows is still a perfectly valid table, so the chain never breaks.

Q: operation that breaks closure on square matrices. (c) Name an operation that preserves closure on colors [r,g,b].A: Breaks it on square matrices: the determinant (returns a scalar, not a matrix); also eigenvalue decomposition or rank. (Note: inverse does not break it - an invertible matrix's inverse is still a square matrix.) (c) Preserves it on colors: averaging two colors (blur), or addition with clipping to 0–255, or scalar brighten/darken with clipping - all

return another [r,g,b]. (Grayscale breaks it - collapses to a single value.)

8 Operators: Two are unary (one table in): SELECT (σ) keeps a subset of rows; PROJECT (π) keeps a subset of columns. Six are binary (two tables in): UNION, INTERSECT, DIFFERENCE, PRODUCT (Cartesian every row of A paired with every row of B), JOIN (combine on a matching column), and DIVIDE (rows of A that match all of B "students who took every CS course"). UNION, INTERSECT, and DIFFERENCE are literally set operations on the rows: $A \cup B$ (in either), $A \cap B$ (in both), $A - B$ (in A but not B). Picture a Venn diagram. They require the two tables to be union-compatible (same columns/types).

Joins: inner/natural join keeps only matching rows; a left / right / full outer join also keeps the unmatched rows from one or both sides, filling the gaps with nulls.

Inner join>Returns only matching rows from both tables.

Left or left outer join>Returns all rows from the left table and matching rows from the right table.

Right or right outer join>Returns all rows from the right table and matching rows from the left table.

Full outer join>Returns all rows from both tables.

Cross join: Returns the Cartesian product/Every combination of rows

Normalization: cleanup process that removes redundancy from a table, because redundancy is what causes anomalies situations where your data breaks when you insert(you can't add a fact without unrelated data ex: can't record a new project until someone is assigned to it.), update(the same value is stored in many rows, so if you change it in one place and miss another, the data contradicts itself.), or delete(deleting one row accidentally erases an unrelated fact (delete the last assignment and the project's name vanishes too)).

2 bad dependencies you check for - **partial dependency** is when non-key column depends on only part of a composite key (only possible when the key has multiple columns).

transitive dependency is when a non-key column depends on another non-key column a regular attribute "acting like a key." Ex: construction company table: PROJ_NUM, PROJ_NAME, EMP_NUM, EMP_NAME, JOB_CLASS, CHG_HOUR, HOURS.

0NF $\rightarrow$ 1NF: fill in all the blanks (no repeating groups, a value in every cell) and identify the primary key -here the composite (PROJ_NUM, EMP_NUM). Now it's a real table, but it still has bad dependencies.

1NF $\rightarrow$ 2NF: remove partial dependencies. PROJ_NAME depends only on PROJ_NUM (half the key), so split it out $\rightarrow$ a PROJECT table, an EMPLOYEE table, and an ASSIGN table for (proj, emp, hours).

2NF $\rightarrow$ 3NF: remove transitive dependencies. CHG_HOUR depends on JOB_CLASS, which depends on the employee -a non-key determining a non-key. Promote it into its own JOB(JOB_CLASS, CHG_HOUR) table; JOB_CLASS stays as a foreign key.

BCNF tightens 3NF: every determinant must be a candidate key. **denormalization** is the deliberate opposite- re-adding some redundancy to speed up reads (fewer joins), common in reporting/warehouse tables. Higher normal form = cleaner data but more joins; it's a trade-off.

SQL - declarative you say what you want, not how to get it splits into **DDL** (define structure: CREATE, ALTER, DROP), **DML** (work with data: INSERT, SELECT, UPDATE, DELETE), and **TCL** (COMMIT, ROLLBACK)

DELETE vs DROP: Both wipe all the rows. The difference: DELETE FROM product; empties the table but keeps its structure (columns stay, you can re-insert). DROP TABLE product; deletes the whole table definition you'd have to CREATE TABLE all over again

SELECT cols FROM t WHERE row-condition GROUP BY c HAVING group-condition ORDER BY c. Order of execution: FROM $\rightarrow$ WHERE $\rightarrow$ GROUP BY $\rightarrow$ HAVING $\rightarrow$ SELECT $\rightarrow$ ORDER BY.

conditions are where the meaning lives: comparisons and AND/OR/NOT; LIKE 'A%' (starts with A, %=any chars, _=one char); BETWEEN, IN (...), IS NULL. computed column like SELECT qty*price AS total is "feature engineering" a new column derived on the fly.

make it FAIL: classic trick: WHERE pid = 3 AND pid = 5. Since pid is a primary key, no single row can be both 3 and 5, so the query returns zero rows.

ON DELETE CASCADE means changes to a parent ripple to its children - delete a VENDOR and its PRODUCTS go too. The order matters, which is why he mentions "cascade ordering."

Aggregate functions - COUNT, SUM, AVG, MIN, MAX - collapse many rows into a single number. aggregate like AVG(price) with no grouping gives "the same value for the whole table"- one number summarizing everything. GROUP BY then chops the rows into categories and runs the aggregate per category; he reads it as "itemize/categorize by." GROUP BY with no aggregate is meaningless.

Why can't WHERE use aggregate functions but HAVING can? Because WHERE runs before GROUP BY and aggregation, while HAVING runs after groups and aggregates have been created. WHERE vs HAVING: WHERE filters rows before grouping; HAVING filters groups after aggregating. "Vendors whose products average over \$50" needs HAVING, because the average doesn't exist until after you group.

subquery can sit in the WHERE, the FROM (used as a temporary table), or the SELECT.

correlated subquery is the trickier one: the inner query re-runs for every row of the outer query because it references that row (e.g. "products priced above their own vendor's average").the inner query references a column from the outer query, so it's re-evaluated once per outer row.

> **ALL()** means greater than the max; > **ANY()** means greater than the min.

view is a saved query that behaves like a virtual table - a "window" onto the data, not stored rows.

sequence auto-generates numbers (NEXTVAL hands you the current value, then increments).

PL/SQL adds procedural features (variables, IF, loops) in DECLARE/BEGIN/END blocks.

trigger is code that fires automatically before/after an INSERT/UPDATE/DELETE- e.g. after a sale, auto-decrement stock.

stored procedure performs an action;

stored function returns a value.

transaction is one logical unit of work- a bundle of reads and writes that must succeed or fail together, taking database from one consistent state to another. guarantees are **ACID**.Without TM you lose ACID. Concretely: lost updates (two transactions overwrite each other), uncommitted/dirty reads, inconsistent retrievals, and no serializability - plus a crash leaves the DB in a half-finished, unrecoverable state. The two extremes: do nothing → fast but corrupt/unreliable data; vs. full TM → correct but with locking overhead.

Atomicity- all-or-nothing. A bank transfer debits one account and credits another; if the credit fails after the debit, you roll back both, as if nothing [happened.Ex](#):where it is violated Alice has \$100, Bob \$50. Transaction T1 transfers \$20 in two steps - subtract \$20 from Alice (→ \$80), then add \$20 to Bob (→ \$70). Transaction T2 is an administrator querying the sum of both balances. With atomicity, T2 must always see \$150. Atomicity is violated if T2 observes the **mid-flight state** - after step 1 but before step 2 - where the sum reads \$130. Equivalently: if the system crashes after step 1 and the debit is left applied without the credit, money has vanished. Atomicity means the whole transaction commits or none of it does.

Consistency- integrity rules are never violated; the total money in the system is the same before and after the transfer.

Isolation-concurrent transactions behave as if each ran alone; two people buying the last seat don't both succeed.

Durability- once you commit, the change survives a crash; a power outage can't undo it.

three concurrency problems happen without isolation.: Lost update- two transactions read the same value, both write, one overwrites the other. Uncommitted / dirty read-reading data another

transaction wrote but hasn't committed (and may roll back). Inconsistent retrieval -a reader (e.g. a report generator) reads some rows before and some after another transaction's update, so an aggregate comes out wrong. This is exactly the report-mid-transfer scenario.

COMMIT makes changes permanent.

ROLLBACK discards them (a mid-way crash rolls back automatically).

How isolation is enforced? Through locking. Db has lock manager. **shared (read) lock** can be held by many transactions at once (reading doesn't change anything). **write lock** is exclusive- only one transaction, and all read locks are revoked first so nobody reads half-finished data.

Locking like this is called **pessimistic** (assume a conflict will happen, so lock first). opposite is **optimistic** -don't lock at all, used when you're sure no one else will touch the data.

deadlock is the trap that locking can create: transaction A holds a lock that B wants, and B holds a lock that A wants- so both wait forever. (circular wait)

Three strategies for deadlock prevention:1.Detection (let it happen, then break it). Allow deadlocks, but periodically build a wait-for graph — a node per transaction, an arrow from $T_i \rightarrow T_j$ if T_i is waiting for a lock T_j holds. A cycle = a deadlock. To remove it, the DBMS aborts and rolls back a "victim" transaction to break the cycle, then restarts it later. This is the one that literally removes an existing deadlock. 2. Prevention (make a cycle impossible). Structure the locking so a circular wait can never form. Common ways:Lock ordering — every transaction acquires resources in the same global order, so cycles can't arise. Acquire all locks up front before starting (all-or-nothing). Timestamp schemes — wait-die / wound-wait: based on transaction age, a transaction is forced to wait or is rolled back rather than risk a cycle.3. Avoidance / timeouts (practical middle ground). Use a lock timeout — if a transaction waits too long for a lock, assume deadlock, abort and retry.

Two-phase locking(concurrency control, ensures serializability, single node-grow/shrink locks; for single node) ties in here: a transaction first only acquires locks-the growing phase-then only releases them-the shrinking phase never going back to acquire after it has started releasing.Deadlock example: T1 locks X, T2 locks Y; then T1 asks for Y (held by T2) and T2 asks for X (held by T1). Each waits for the other → circular wait → deadlock. (Detect via a wait-for-graph cycle, then abort a victim.)

To **keep a transaction atomic** across sites, you use **two-phase commit (2PC)**(distributed commit, ensures atomic commit; for entire collection of nodes in a distributed db): in phase 1 a coordinator asks every site "are you ready to commit?" (they vote); in phase 2, if everyone said yes it tells them all to commit, otherwise it tells them all to abort. All-or-nothing, across machines.

Common btw 2PL, 2PC - both are used to maintain correctness when multiple resources or nodes are involved that need to carry out multiple concurrent transactions

CAP theorem (Brewer): Among Consistency, Availability, and Partition-tolerance, you can only fully guarantee two of three. Network partitions will happen, so in practice systems must choose, during a partition, between C and A. Banking picks C (better to refuse than show a wrong balance); a social feed picks A (better to show slightly stale likes than an error).

Q: vendor stores customers in one flat table with columns including firstName, lastName, address, city, state, zip, phone, skypeID, email, and several rows clearly belong to the same household (same address, blank phones). Name two problems and how you'd fix them. **A:**No primary key / no unique identity: two people can share a name and address, so rows aren't uniquely identifiable. Fix: add a surrogate customer_id PK.Redundancy + multivalued/optional data: shared household address is repeated (update anomaly), and contact fields like phone/skype are sparse (lots of NULLs / a customer may have several). Fix: split into CUSTOMER and a related CONTACT (or ADDRESS) table linked by FK -normalizing away the repetition and the NULLs.

Q: Fake news: $T_1 + T_2$ -there's no "addition" operator (combining rows is UNION, written $\cup$, with union-compatibility rules; "+" isn't it). $T_1 - T_2$ (DIFFERENCE): rows in T_1 that are not in T_2 . If $T_1=\{1,2,3\}$ and $T_2=\{2\}$, then $T_1-T_2=\{1,3\}$. $T_1 \times T_2$ (PRODUCT): Cartesian product- each row of T_1 paired with every row of T_2 (so 2 rows $\times$ 3 rows = 6 rows, columns combined). ($T_1 \div T_2$ is DIVIDE- rows of T_1 matching all of T_2 .) Note all of them are closed: table in, table out.

Q:SELECT name FROM driver WHERE vehicle IN (SELECT vehicle FROM vehicles) GROUP BY name HAVING COUNT(*) = (SELECT COUNT(*) FROM vehicles); this implements division alg operation it return drivers associated with every vehicle in vehicles

Q:SELECT COUNT(*) FROM books; → 100. ... WHERE author_id = '1'; → 15. What does SELECT COUNT(*) FROM books WHERE author_id <> '1'; return? (a) 85 (b) 15 (c) insufficient information.**A:**Insufficient information. Rows with a NULL author_id satisfy neither = '1' nor <> '1', so if any author_ids are NULL the answer is less than 85. This is the NULL/three-valued-logic gotcha.

Q:You transfer \$7,200 from savings to checking on your phone. Mid-transfer, a monthly-statement report generator starts reading both balances. What can go wrong, and what is the scenario called?**A:** report can read a mid-flight state - money already subtracted from savings but not yet added to checking - so the totals are wrong. This is an inconsistent retrieval (a violation of isolation/atomicity; the reader sees an intermediate, never-committed state). Proper locking/isolation prevents it.

Q:File systems suffered from structural dependency. Define it with a brief example. Then: the relational model appears to make it vanish

- but it doesn't. Where does it go? **A:**Definition: program logic is tied to the physical/positional layout of the data; change the layout and every program breaks. Example: a payroll file stores EmpID in bytes 1–5, Salary in 6–11; insert a MiddleName field and every routine that assumed Salary starts at byte 6 breaks.

Where it goes: up to the logical schema / SQL / application layer. Renaming/reordering columns, changing datatypes, or splitting tables still breaks queries, stored procedures, and ORM/app code that reference the old structure. Views can hide some of it; otherwise the dependency now lives in the application layer instead of the file layout.

JOIN vs EXISTS vs IN: Three queries summing price per product - one with JOIN PRODUCTS, one with WHERE EXISTS (SELECT 1 FROM PRODUCTS...), one with WHERE product_id IN (SELECT product_id FROM PRODUCTS). All three are identical provided product_id is the unique PK of PRODUCTS and referential integrity holds: EXISTS and IN both just test "a matching product exists," and the JOIN doesn't inflate the sum because each sale matches exactly one product. The trap to mention: if PRODUCTS had duplicate product_ids, the JOIN would multiply rows and change the SUM, while EXISTS/IN would not - then they'd differ.**EXISTS** checks for existence of rows returned by subquery. It returns true if subquery returns atleast one row otherwise false. If exists return false, the main query will be skipped (only when). cares about existence, not values, and short-circuits (stops at the first match).

FROM+JOIN-correspond to iteration across collections. Implemented as nested loops(outer loop over rows of first table, inner loop over rows of second) equivalent to nested for loops

Where conditional filter if statement keep rows if predicate is true

Select - set oriented, non procedural

"What is a vector database and why does it exist?" · "What's an embedding?" · "How is searching a vector DB different from a relational query?" · "Why can't a normal relational DB do this well?" Embedding = turn text / image / audio / video into a fixed-length numeric vector (list of numbers) that captures its meaning/semantics. A vector DB stores these high-dimensional vectors. Query type = nearest-neighbor / similarity search ("find similar"), not exact match — using a distance/similarity metric (cosine similarity, Euclidean). Often approximate nearest-neighbor (ANN) search for speed. Powers semantic search and RAG (retrieval-augmented generation) — i.e. an AI agent's "memory"/retrieval. It's the newest entry in the data-model evolution (hierarchical → network → relational → ... → vector). **Why relational falls short:** tables do exact-match lookups on rows; they can't natively ask "what's similar in meaning."

"What is text-to-SQL?" · "How do AI agents use databases / why is SQL 'going away'?" · "Why does this matter?" Text-to-SQL (natural-language-to-SQL): describe the request in

plain English → an LLM generates the SQL. Example: "vendors in CA with more than 5 products" → the model writes the SELECT. You prompt instead of hand-writing SQL; SQL is being abstracted away. Agent = an LLM that can plan and take actions / use tools on its own — it can spin up a database, run queries, and tear it down, generating and executing the SQL itself. That's why you're tested on reading/reasoning about SQL, not writing it — humans now verify what the agent produces. Tokens = the chunks of text an LLM processes; a "token budget" = how much it can read/generate (cost). These two topics connect: embeddings + a vector DB give the agent its memory/retrieval (RAG). agents can hallucinate / write wrong queries, so you need human verification, schema awareness, and there are security risks (e.g. prompt injection).

Set operations: union ($\cup$) = in either; intersection ($\cap$) = in both; difference ($-$) = in A but not B; complement ($\neg A / A'$) = everything in the universe U not in A. must define the universal set U before you can take a complement.

De Morgan's laws: $\neg(A \cup B) = \neg A \cap \neg B$ and $\neg(A \cap B) = \neg A \cup \neg B$.

Ties to dbs: relational-algebra operators UNION, INTERSECT, DIFFERENCE are these set operations (require union-compatible tables); SQL: UNION, INTERSECT, EXCEPT/MINUS.

Set operations are closed (set in → set out) — a closure

"What is NoSQL and why did it arise?" · "Distributed processing vs distributed database?" · "What is the CAP theorem?"NoSQL = "not only SQL" — non-relational, flexible/schema-less, built for huge scale and unstructured/semi-structured data. Four types: document (JSON — e.g. MongoDB), key-value, column-family, graph.

Why it arose: relational limits — rigid schema, can't store nested/multivalued data in a cell (the JSON fix), and hard to scale horizontally. ACID vs BASE: NoSQL often trades strict ACID for BASE (Basically Available, Soft state, Eventual consistency).

Cloud DB = hosted, elastic/scalable, managed AWS (DynamoDB = NoSQL, Aurora/RDS, S3). Distributed processing = many CPUs, one DB; distributed database = the data itself spread across sites/nodes. Fragmentation (horizontal = rows, vertical = columns) and replication (copies at sites); goal = transparency (user thinks it's one local DB).

"Why move databases to the cloud?" "Moving to the cloud means databases become on-demand as the professor noted, an agent can spin one up in seconds, use it, and tear it down. You get elastic scaling (the only way to handle an Amazon-sized DB of half-a-billion customers and 20M products), pay-as-you-go cost, and a managed, highly-available service that runs on your behalf with no hardware to maintain at the cost of some control and vendor lock-in."

Relationship examples: 1:1: PERSON ↔ PASSPORT One person has one passport.One passport belongs to one person.COUNTRY

↔ CAPITAL_CITY One country has one capital city. One capital city belongs to one country.

EMPLOYEE ↔ PARKING_SPACE One employee is assigned one parking space. One parking space is assigned to one employee.

1:M: DEPARTMENT → EMPLOYEEOne department has many employees.An employee belongs to one department. CUSTOMER → ORDEROne customer can place many orders.An order belongs to one customer.COUNTRY → STATEOne country has many states/provinces.A state belongs to one country.

M:N: STUDENT ↔ COURSEA student can take many courses.A course can have many students.AUTHOR ↔ BOOKAn author can write many books.A book can have multiple authors.DOCTOR ↔ PATIENTA doctor treats many patients.A patient may visit many doctors.

SQL-coding relations:FROM = the loop over a table's rows (for row in table)FROM + JOIN = nested for-loops (outer table × inner table)WHERE = if (keep the row only if the condition is true)SELECT cols = the output line (emit only those columns)GROUP BY = put rows into a dictionary/hashmap keyed by the group columnHAVING = if, but applied after grouping (filter the groups)ORDER BY = sort() the resultDISTINCT = set() / remove duplicatesCOUNT/SUM/AVG/MIN/MAX = an accumulator loop (total += x, count++)LIMIT / TOP n = break after n / slice list[:n]INSERT = append a row to the listUPDATE = loop + assignment (change rows in place)DELETE = remove/filter out rowsSubquery = a function call (inner runs first, returns to outer)Correlated subquery = a function called inside the loop, re-run once per rowEXISTS = any() — true if at least one matchIN (...) = membership test (x in list)ALL = greater than the max; > ANY = greater than the min (all()/any())UNION / INTERSECT / EXCEPT = set union / intersection / differenceAND / OR / NOT = boolean && / || / !LIKE = string match / regex

CASE WHEN = if / elif / else (switch)Outer join = nested loop that keeps unmatched rows and fills with NULL.CREATE TABLE = define a class/struct; DROP TABLE = delete that definitionVIEW = a saved function / virtual table (not stored)Sequence / NEXTVAL = an auto-increment counter (i++)COMMIT / ROLLBACK = save/return vs discard/undo (try/except)

Execution order: loop → filter → group → aggregate → having → sort → project.